
Análisis y Diseño Orientado a Objetos

Introducción

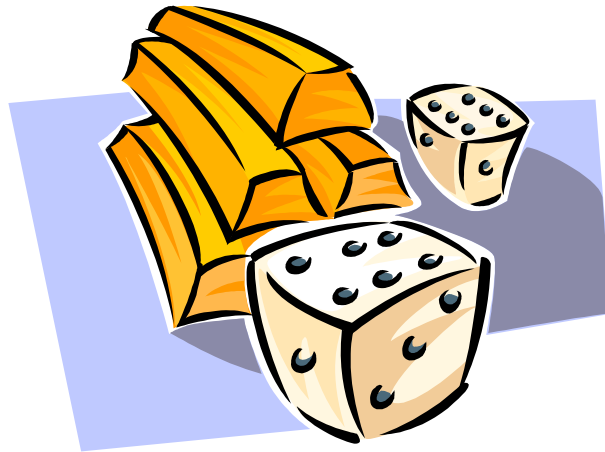


- **Proceso de desarrollo de software:**
 - Forma disciplinada de asignar tareas y responsabilidades en una empresa de desarrollo (quién hace qué, cuándo y cómo).
- **Objetivos:**
 - Asegurar la producción de software de calidad dentro de plazos y presupuestos predecibles.

Introducción

Ejemplo: *Un juego de dados.*

Se tiene un juego de dados en que un jugador lanza dos dados. Si el total obtenido es siete, el jugador gana, en caso contrario pierde.



Introducción

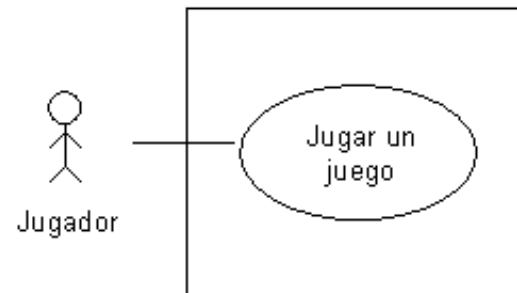
1. Definición de casos de uso

Los *casos de uso* son descripciones narrativas en lenguaje natural de los procesos del dominio en un formato estructurado de prosa. Describen una secuencia de acciones.

Caso de uso: Jugar un juego.

Participantes: Jugador.

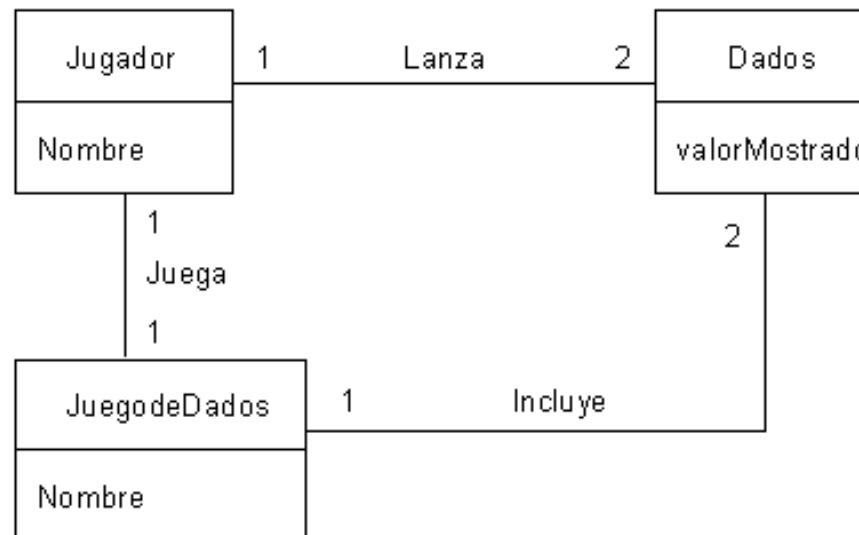
Descripción: Este caso de uso comienza cuando el jugador recoge y lanza los dados. Si los puntos suman siete, gana y pierde si suman cualquier otro número.



Introducción

2. Definición de un modelo conceptual

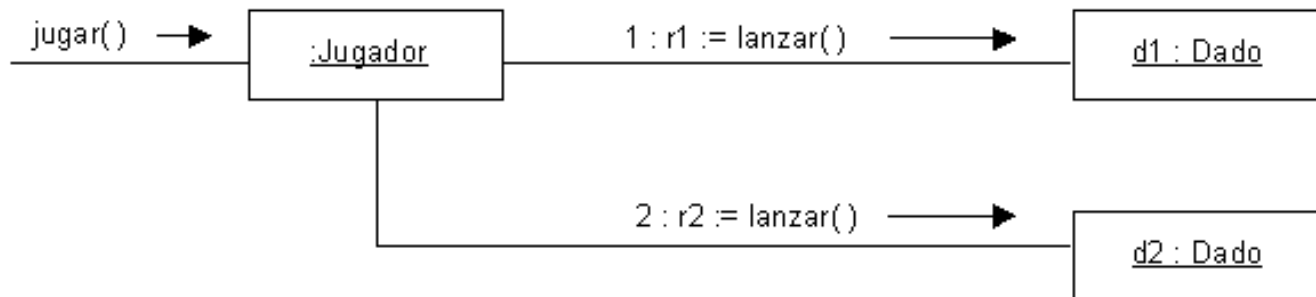
Un *modelo conceptual* muestra gráficamente los conceptos (clases de objetos), los atributos y las asociaciones más importantes del dominio del problema. Supongamos que queremos hacer una simulación del juego de dados:



Introducción

3. Definición de los diagramas de colaboración

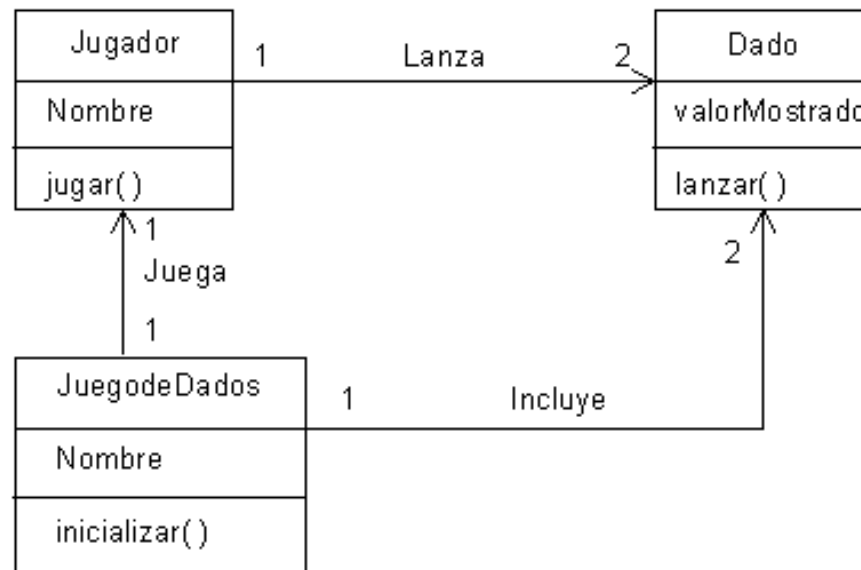
Los *diagramas de colaboración* representan el flujo de mensajes entre las instancias y la invocación de métodos.



Introducción

4. Definición del diagrama de diseño de clases

¿Cómo se relacionan unos objetos con otros?, ¿cuáles son las características (métodos y atributos) de cada clase?



Introducción

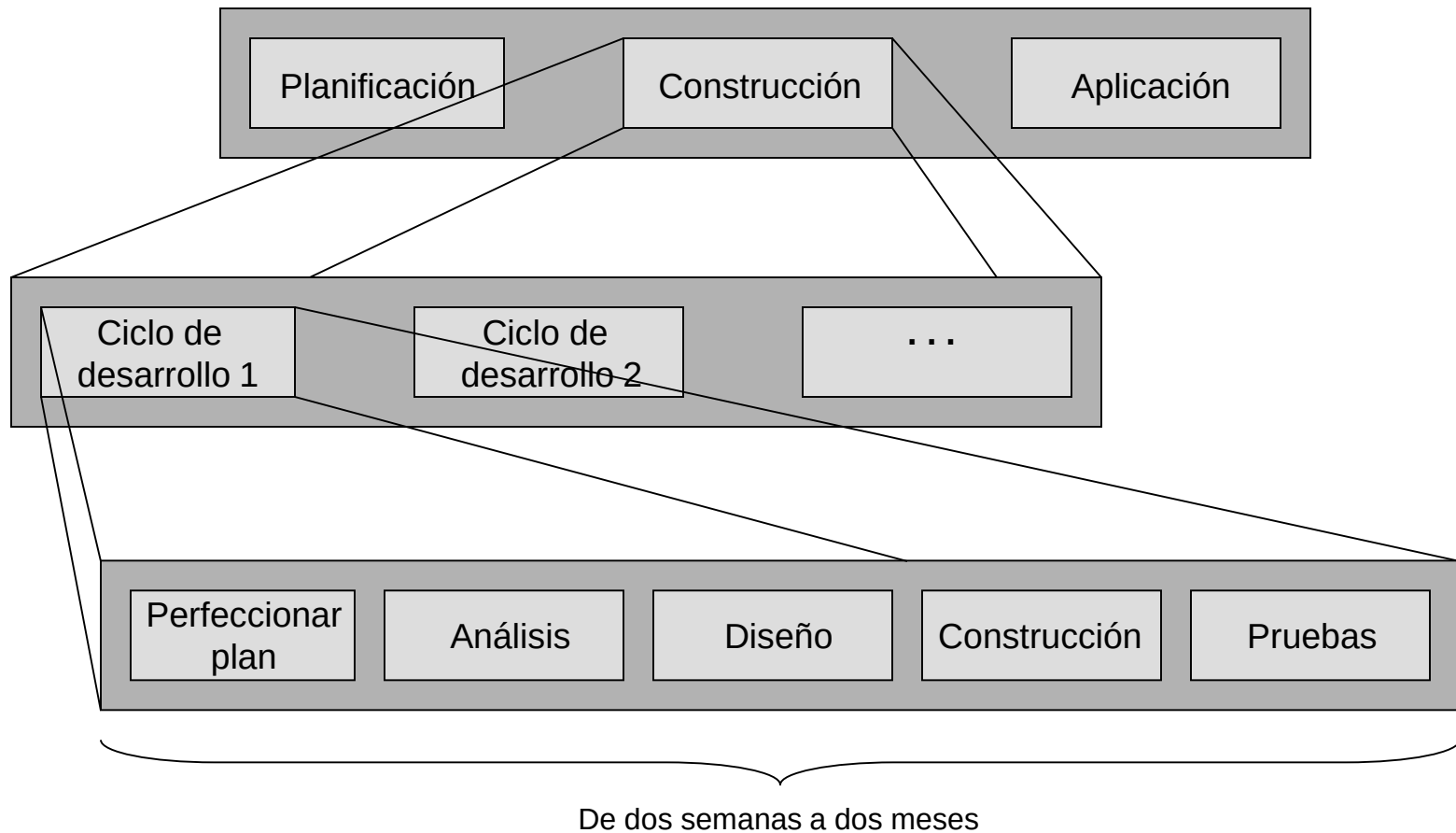
5. Codificación

Escritura del código en un lenguaje orientado a objetos.

```
class JuegodeDados {
    String Nombre;
class Jugador {
    String nombre;
    public Jugador(String nombre) {
        this.nombre = nombre;
    }
    public jugar(Dado d1,d2);
        int dado1 = d1.lanzar();
        int dado2 = d2.lanzar();
    }
}
public void Dado(){
    int ValorMostrado;
    public Dado {
        this.ValorMostrado = 0;
    }
    public lanzar();
        this.ValorMostrado = Math.random(1,6);
    }
} ...
```

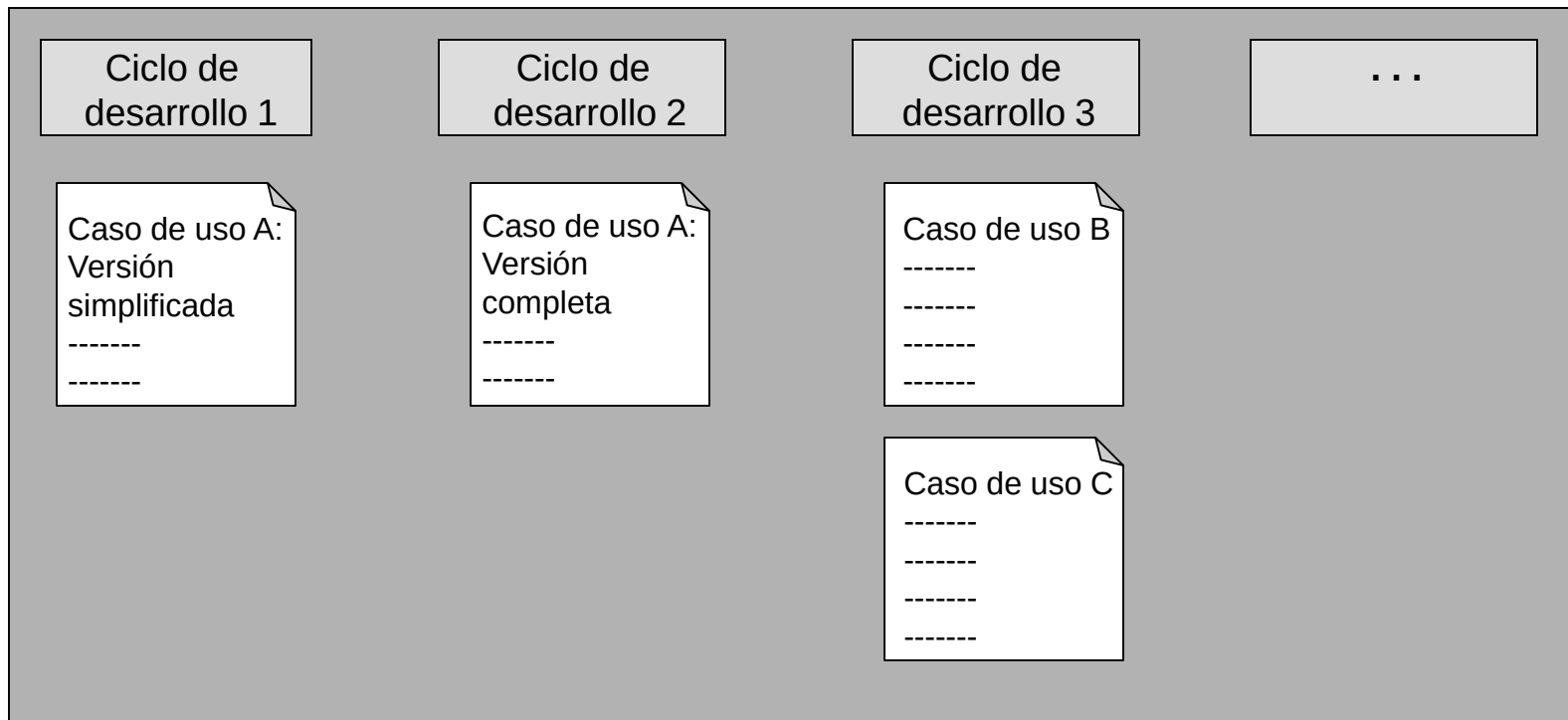
Introducción

Proceso de desarrollo de software



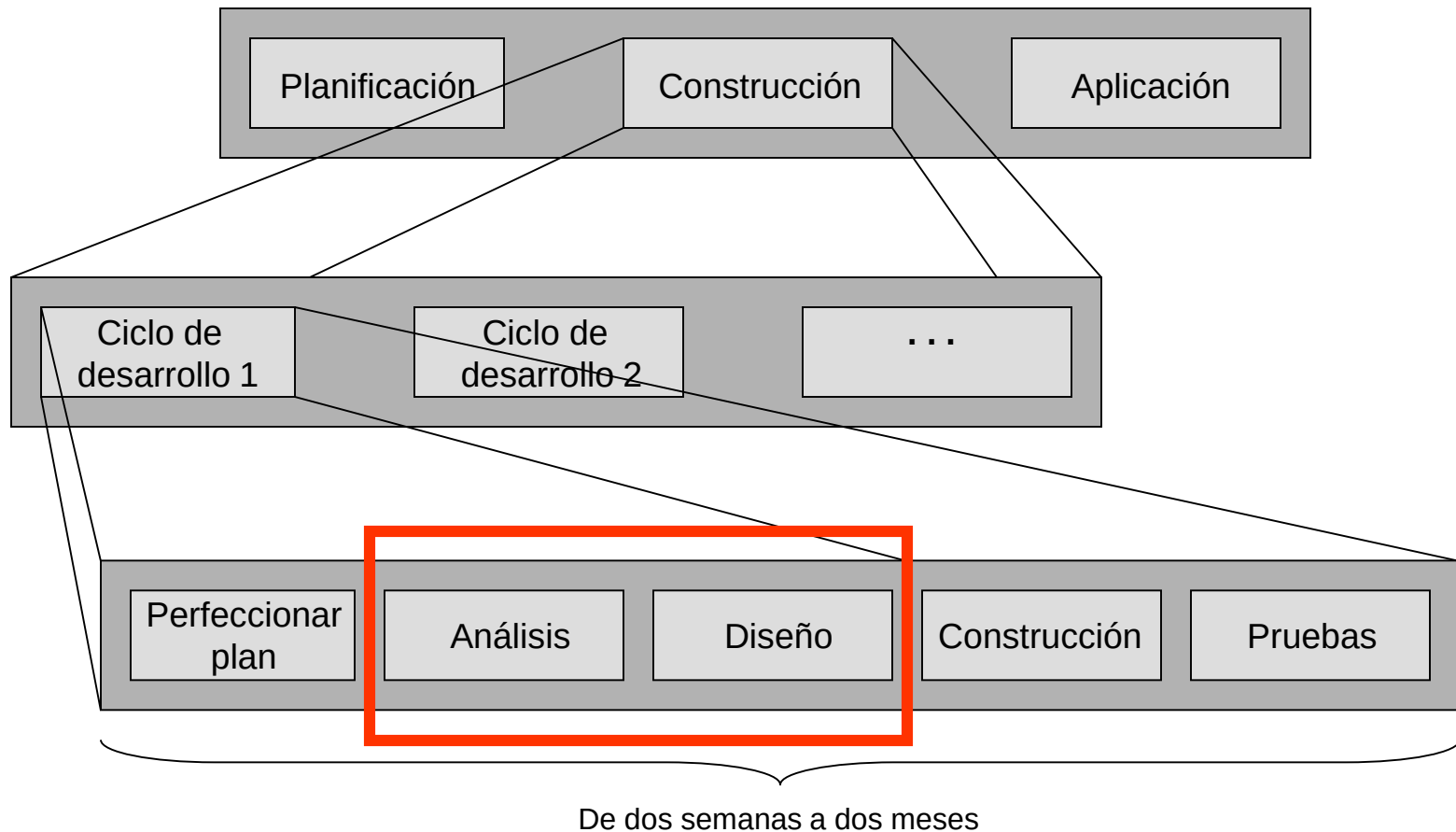
Introducción

Proceso de desarrollo de software



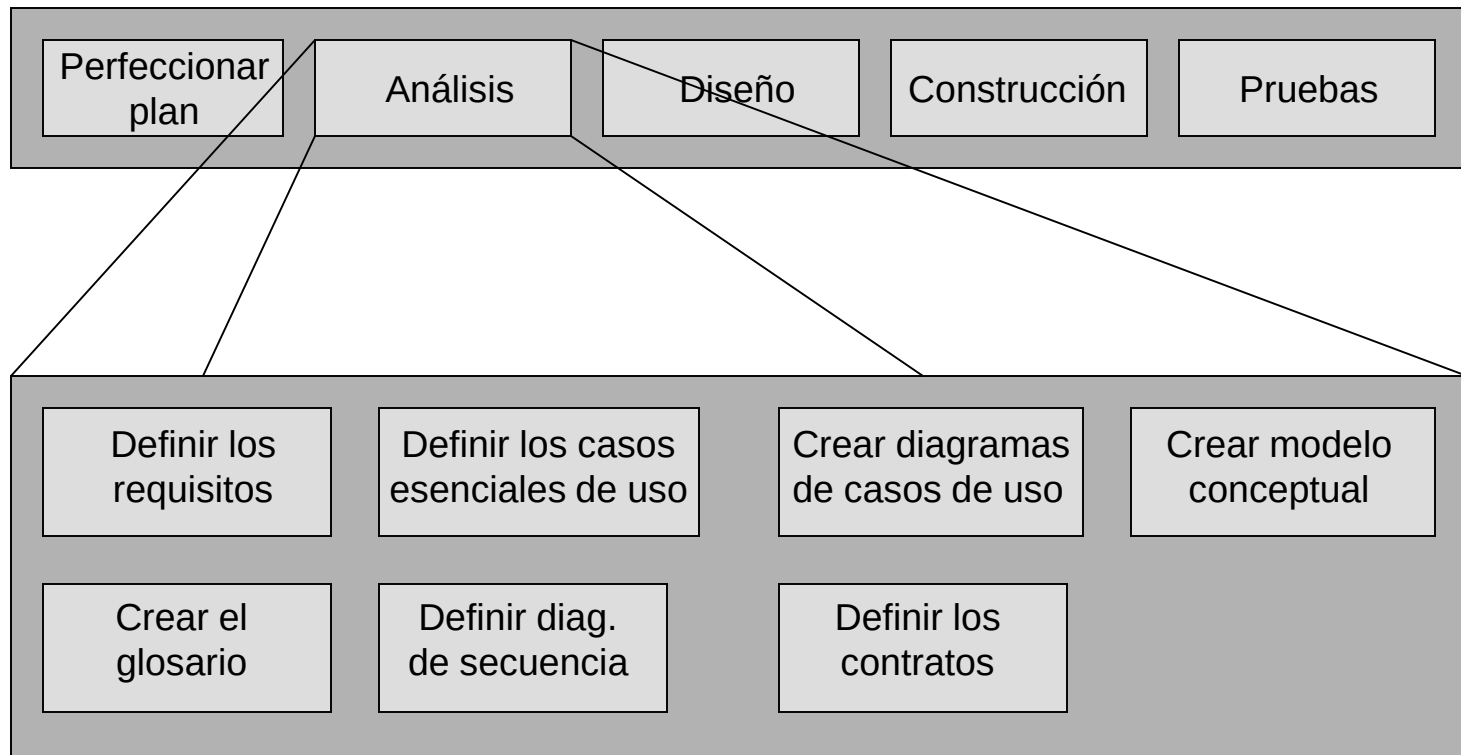
Introducción

Proceso de desarrollo de software



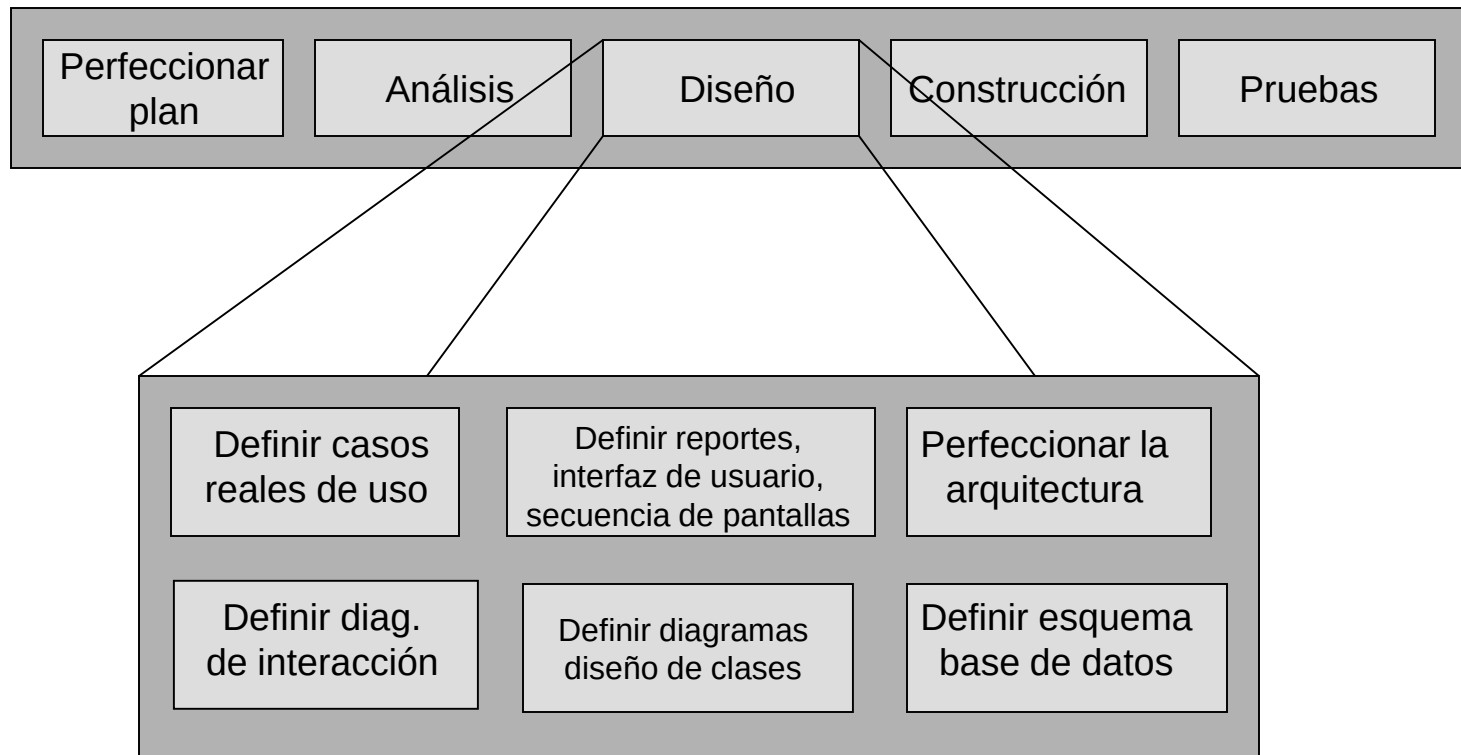
Análisis y Diseño OO

Algunas de las tareas a realizarse en la etapa de análisis (dominio del problema) son las siguientes:



Análisis y Diseño OO

Algunas de las tareas a realizarse en la etapa de diseño (dominio de la solución) son las siguientes:



Caso de estudio

Caso de estudio: *punto de venta*

Supongamos como caso de estudio el sistema de una terminal de punto de venta. Esta terminal es un sistema automatizado con el que se registran las ventas y se realizan los pagos.

Por lo general este tipo de sistemas comprenden hardware (un computador y un lector de código barras) y software (el sistema que se ejecuta en la terminal).

Requisitos

Los requisitos

Los requisitos son una descripción de las necesidades o deseos de un producto.

Se recomienda aquí definir al menos los siguientes puntos:

- Panorama general
- Metas
- Funciones del sistema

Requisitos

a) Panorama general

Este proyecto tiene por objeto crear un sistema de terminal para el punto de venta que se utilizará en las ventas de un supermercado.

b) Metas

En términos generales, la meta es una mayor automatización del pago en las cajas registradoras, y dar soporte a servicios más rápidos, más baratos y mejores. Concretamente, la meta incluye:

- Pago rápido de los clientes.
- Análisis rápido y exacto de las ventas.
- Control automático del inventario.

Requisitos

c) **Funciones del sistema**

Las funciones del sistema son lo que éste deberá de hacer.

Las funciones pueden clasificarse en tres categorías: evidentes, ocultas y superfluas. Las evidentes deben realizarse, y el usuario debe saber que se han realizado. Las ocultas también deben realizarse, y puede que no sean visibles para el usuario. Las superfluas son opcionales, y su inclusión no repercute significativamente en el costo ni en otras funciones.

Requisitos

Estas son algunas de las funciones del sistema de punto de venta:

Ref.	Función	Categoría
R1.1	Registra la venta en proceso (actual): los productos comprados.	evidente
R1.2	Calcula el total de la venta actual; se incluye el impuesto.	evidente
R1.3	Captura la información sobre el objeto comprado usando su código de barras, o usando una captura manual del código de producto.	evidente
R1.4	Reduce las cantidades del inventario cuando se realiza una venta.	oculta
R1.5	Se registran las ventas efectuadas.	oculta
R1.6	El cajero debe introducir una identificación y una contraseña para poder utilizar el sistema.	evidente
R1.7	Ofrece un mecanismo de almacenamiento persistente.	oculta
R1.8	Ofrece mecanismos de comunicación entre los procesos y entre los sistemas.	oculta
R1.9	Muestra la descripción y el precio del producto registrado.	evidente

Casos de uso

Casos de uso

Los casos de uso requieren tener al menos un conocimiento parcial de los *requerimientos* del sistema. Un caso de uso es un documento narrativo que describe la secuencia de eventos de un actor (agente externo) que utiliza un sistema para completar un proceso.

Casos de uso

El formato para la descripción de los casos de uso es el siguiente:

Caso de uso: Nombre

Actores: Lista de actores (agentes externos)

Propósito: Intención del caso de uso

Resumen: Repetición del caso de uso de alto nivel o alguna síntesis.

Tipo: Primario, secundario u opcional. Esencial o real.

Referencias

cruzadas: Casos de uso relacionados y funciones relacionadas del sistema.

Descripción: Descripción del caso de uso.

Casos de uso

Ejemplo: el siguiente caso de uso describe el proceso de comprar artículos en una tienda, a través de un terminal de punto de venta.

Caso de uso: Comprar productos

Actores: Cliente, cajero

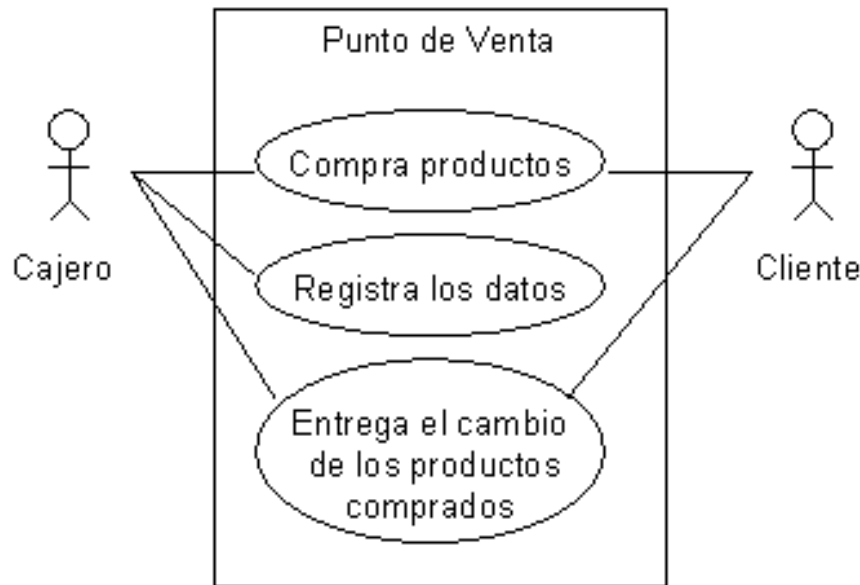
Tipo: Primario

Descripción: Un Cliente llega a la caja registradora con los artículos que va a comprar. El Cajero registra los artículos y cobra el importe. Al terminar la operación, el Cliente se marcha con los productos.

Es conveniente comenzar con los casos de uso de más alto nivel para lograr comprender mejor los principales procesos globales.

Casos de uso

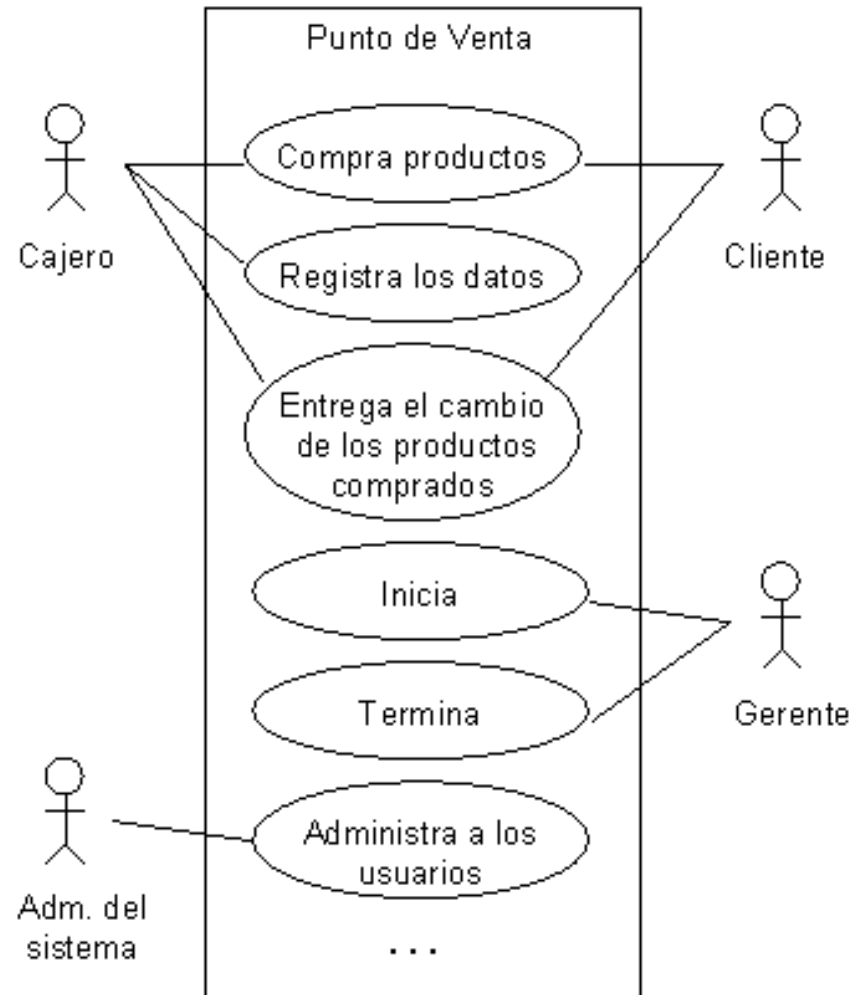
Diagrama UML de casos de uso para el sistema de punto de venta:



Este esquema tiene por objeto ofrecer un diagrama contextual que nos permita conocer rápidamente los actores externos de un sistema y las formas básicas en que éstos lo utilizan.

Casos de uso

Un diagrama de casos de uso más refinado sería el siguiente:



Modelo conceptual

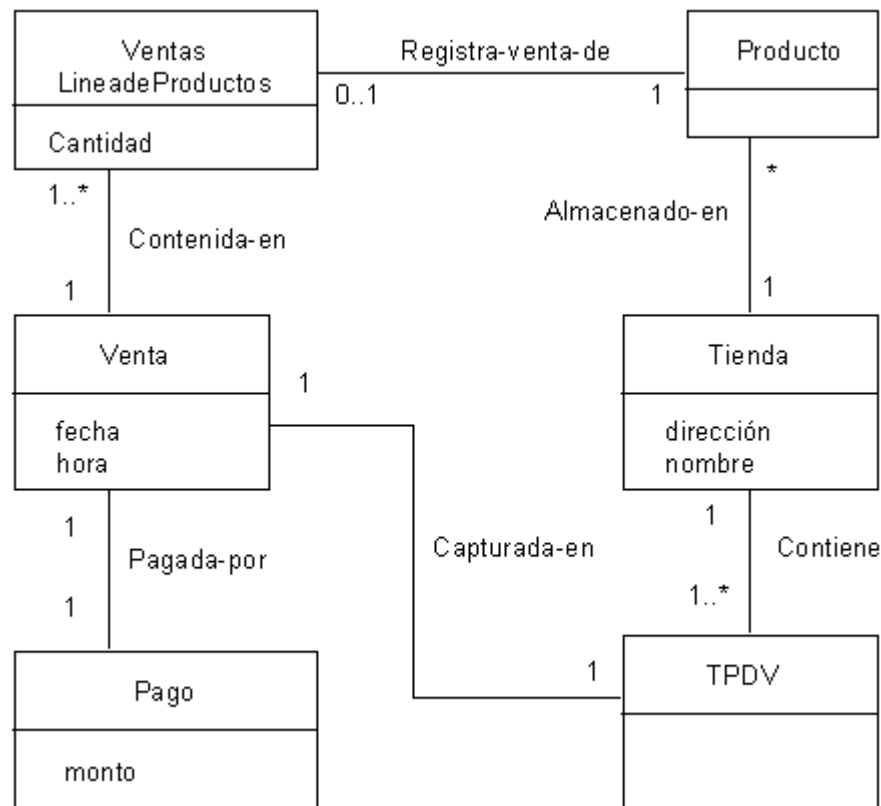
Modelo conceptual

Un modelo conceptual explica los conceptos significativos en un dominio del problema, identificando los atributos y las asociaciones, y es la herramienta más importante del análisis orientado a objetos.

Los casos de uso son una importante herramienta para el análisis de requerimientos, pero realmente no están orientados a objetos. Un modelo conceptual representa cosas del mundo real, no componentes del software. En los diagramas UML se muestran conceptos (objetos), asociaciones entre conceptos (relaciones) y atributos de conceptos (atributos).

Modelo conceptual

La siguiente figura muestra un modelo conceptual parcial del dominio de la tienda y las ventas.



Modelo conceptual

La siguiente lista muestra un conjunto de conceptos idóneos para ser incluidos en el modelo conceptual.

- Objetos físicos o tangibles
- Especificaciones, diseño o descripciones de cosas
- Lugares
- Transacciones
- Línea o renglón de un elemento de transacciones
- Rol de las personas
- Contenedores de otras cosas
- Cosas dentro de un contenedor
- Otros sistemas de cómputo o electromecánicos externos al sistema
- Organizaciones
- Eventos
- Procesos
- Reglas y políticas
- Catálogos
- Registros de finanzas, de trabajo, de contratos, de asuntos legales
- Instrumentos y servicios financieros
- Manuales y libros

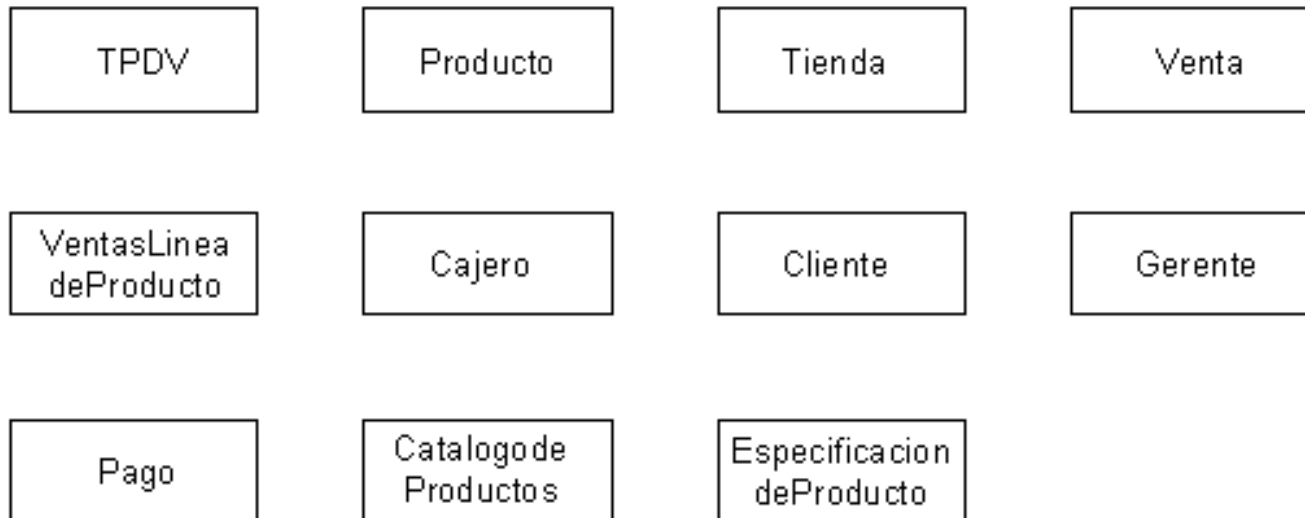
Modelo conceptual

A partir de esta lista de categorías de conceptos podemos generar un conjunto de conceptos para nuestro problema del punto de venta:

TDPV	EspecificaciondeProducto
Producto	VentasLineadeProductos
Tienda	Cajero
Venta	Cliente
Pago	Gerente
CatalogodeProductos	

Modelo conceptual

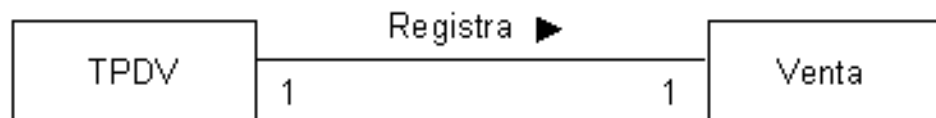
Por tanto, el modelo conceptual inicial del sistema de punto de venta (sin incluir atributos ni asociaciones) sería:



Modelo conceptual

Asociaciones

Una asociación es una relación entre dos conceptos que indica alguna conexión significativa entre ellos. Las asociaciones útiles a determinar, suelen incluir el conocimiento de una relación que ha de preservarse por algún tiempo: puede tratarse de milisegundos o de años (según el contexto). Por ejemplo, ¿debemos recordar cuales instancias de VentasLineadeProducto están asociadas a Venta? Si, porque de lo contrario no sería posible reconstruir la venta, imprimir la boleta ni calcular el total de la venta.



Modelo conceptual

Para identificar las asociaciones más comunes, la siguiente lista es de gran ayuda.

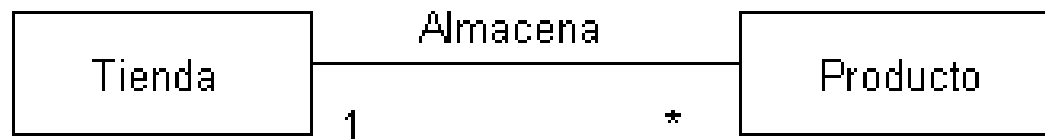
- A es una parte física o lógica de B
- A está lógica o físicamente contenido en B
- A es una descripción de B
- A es un elemento de línea (o renglón) en una transacción o reporte B
- A se conoce/introduce/registra/presenta/captura en B
- A es miembro de B
- A es una unidad organizacional de B
- A usa o dirige a B
- A se comunica con B
- A se relaciona con una transacción B
- A es una transacción relacionada con otra transacción B
- A es propiedad de B

Modelo conceptual

La *multiplicidad* define cuántas instancias de un tipo A pueden asociarse a una instancia del tipo B en determinado momento. Las expresiones de multiplicidad son las siguientes:

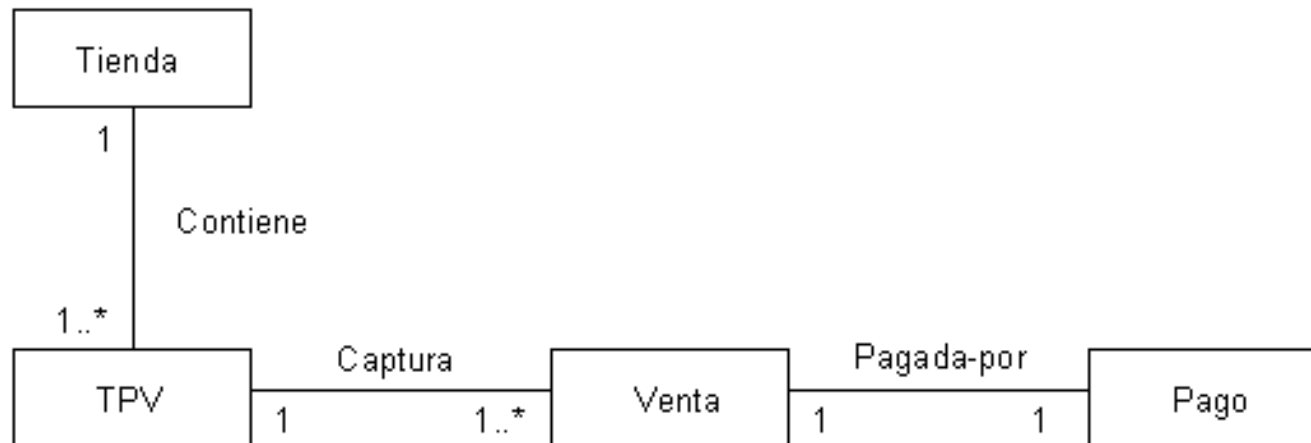
*	cero o más, muchos
1..*	uno o más
1..40	de uno a cuarenta
5	exactamente cinco
2,4,6	exactamente dos, cuatro o seis

Por ejemplo:



Modelo conceptual

Los nombres de las asociaciones deben ser lo más claros posibles, y deben permitir leer y entender fácilmente las relaciones entre conceptos. Por ej.:



Diagramas de secuencia

El *diagrama de secuencia* de un sistema muestra gráficamente los eventos que originan los actores y que impactan al sistema.

La creación de los diagramas de secuencia depende de la formulación de los casos de uso.

Durante la operación del sistema, los actores generan eventos, solicitando alguna operación a cambio. Ejemplo: cuando un cajero ingresa un código de barras de un artículo, está pidiendo al sistema de TPV que registre esa compra. Con este evento se inicia una operación en el sistema.

Diagramas de secuencia

Antes de hacer el diseño lógico de la aplicación de software, es conveniente investigar y definir su comportamiento como una "caja negra".

Se estudia el **comportamiento del sistema**, desde la perspectiva de *qué* es lo que hace, y no de *cómo* lo hace.

Definición: El *diagrama de secuencia* de un sistema es una representación que muestra, en determinado escenario de un caso de uso, los eventos generados por actores externos, su orden y los eventos internos del sistema. En esta fase del proyecto, el sistema mismo es una caja negra.

Diagramas de secuencia

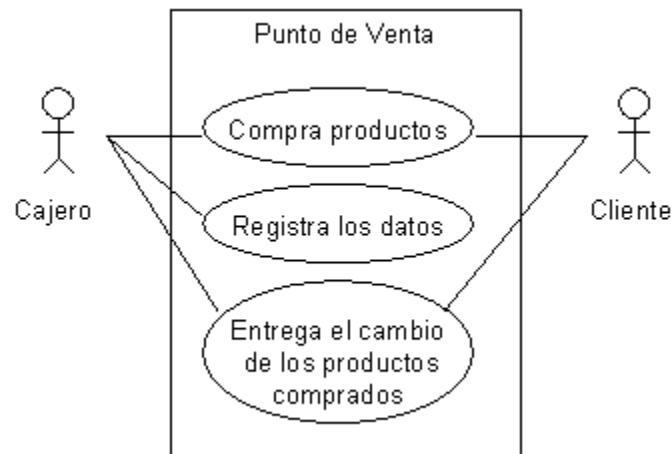
Recordemos el caso de uso Comprar productos:

Caso de uso: Comprar productos

Actores: Cliente, cajero

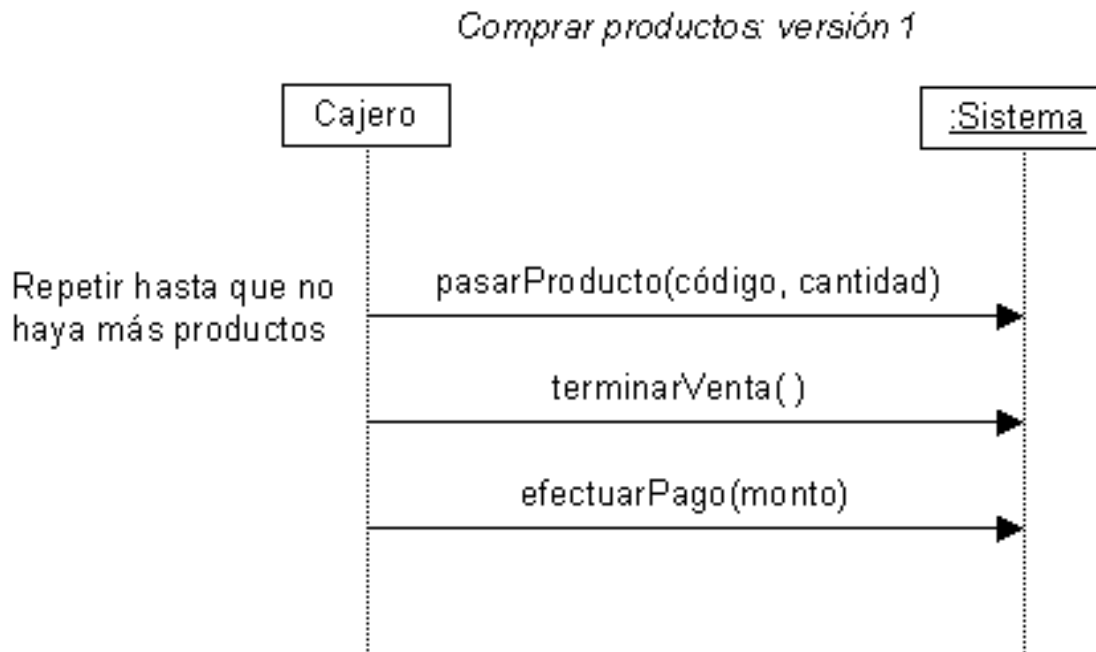
Tipo: Primario

Descripción: Un Cliente llega a la caja registradora con los artículos que va a comprar. El Cajero registra los artículos y cobra el importe. Al terminar la operación, el Cliente se marcha con los productos.



Diagramas de secuencia

El diagrama de secuencia del caso de uso *ComprarProductos* podría ser el siguiente:



Análisis y Diseño OO

Las herramientas usadas en la etapa de análisis (investigación del problema) se pueden resumir en la siguiente tabla.

Herramienta de análisis	Preguntas que contesta
Casos de uso	¿Cuáles son los procesos del dominio?
Modelo conceptual	¿Cuáles son los conceptos, los términos?
Diagramas de secuencia	¿Cuáles son los eventos y las operac. del sistema?

Diagramas de colaboración

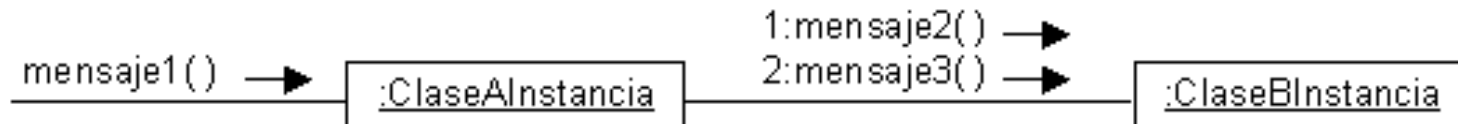
Diagramas de colaboración

Los diagramas de interacción (diagramas de secuencia y diagramas de colaboración) explican gráficamente cómo los objetos interactúan a través de mensajes para realizar las tareas.

Antes de definir estos diagramas, hay que generar el modelo conceptual y los casos de uso reales (estos últimos se generan a partir de los casos de uso definidos en el análisis).

Diagramas de colaboración

Los *diagramas de colaboración* explican gráficamente las interacciones entre las instancias del modelo (objetos). Por ejemplo:



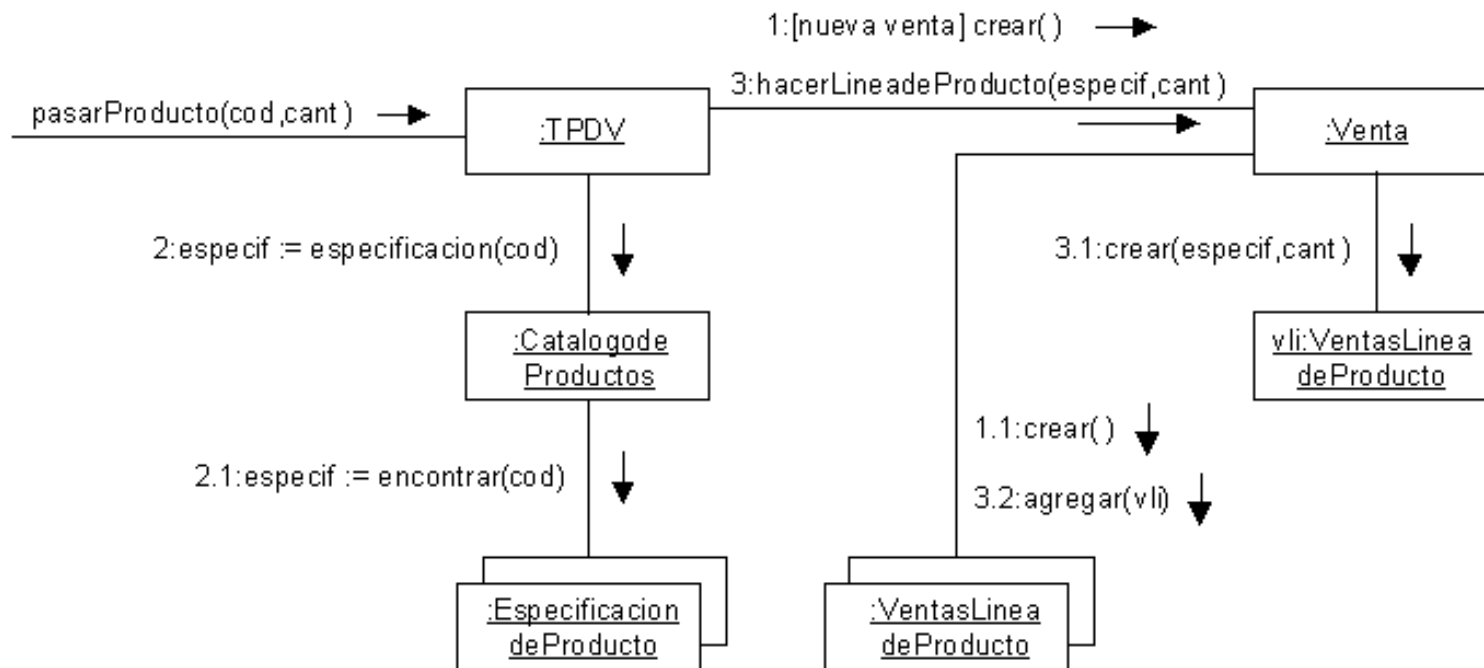
Diagramas de colaboración

Diseño de la solución

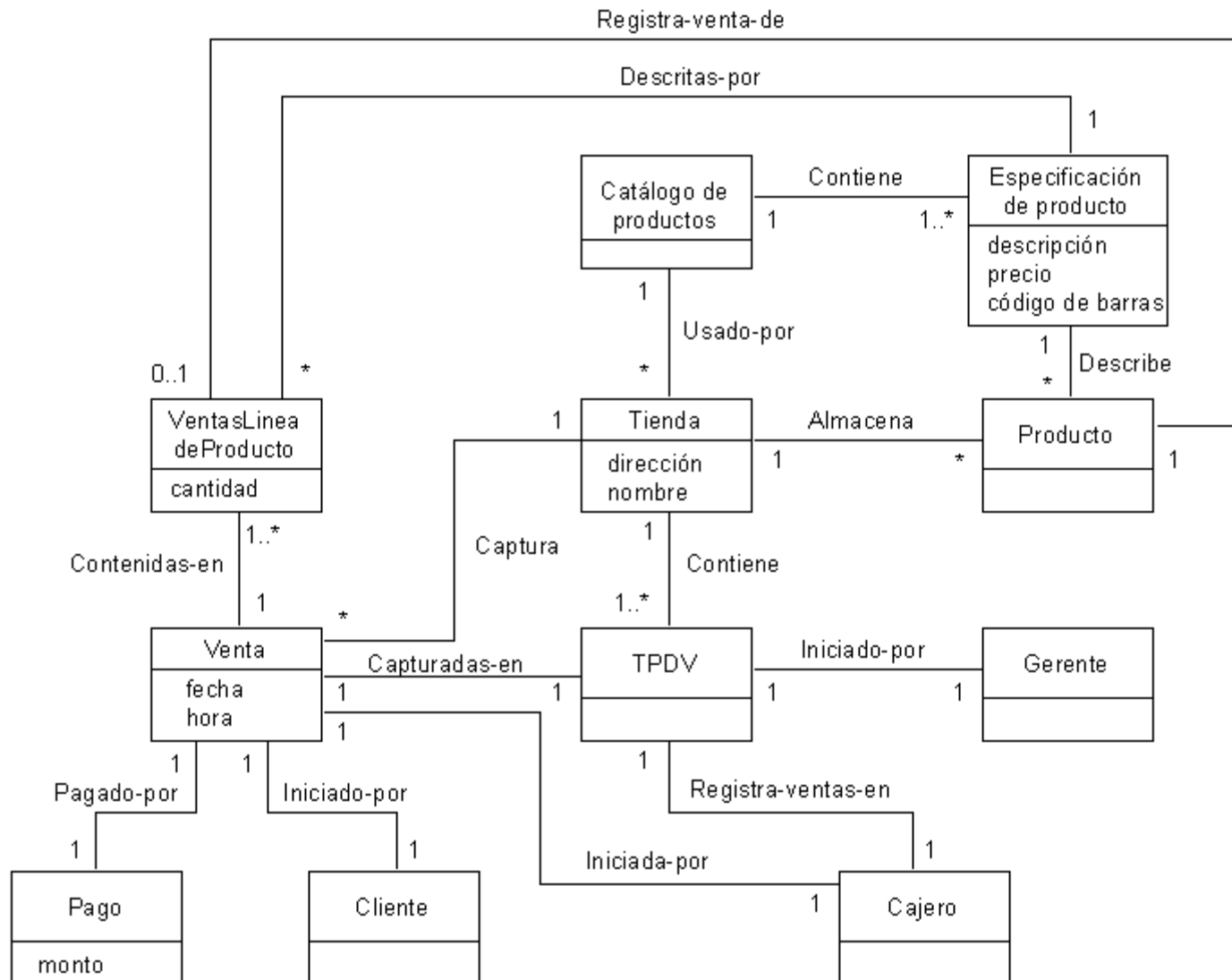
Para cada evento del sistema se debe construir un diagrama de colaboración cuyo mensaje inicial sea el de sus eventos. En el caso del punto de venta, tendremos tres diagramas, uno para cada evento: *pasarProducto*, *terminarVenta*, y *efectuarPago*.

Diagramas de colaboración

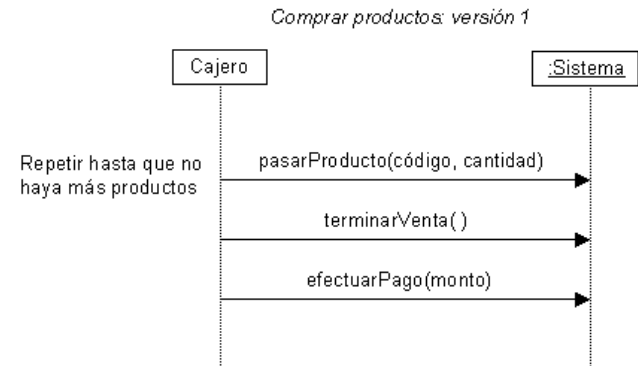
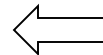
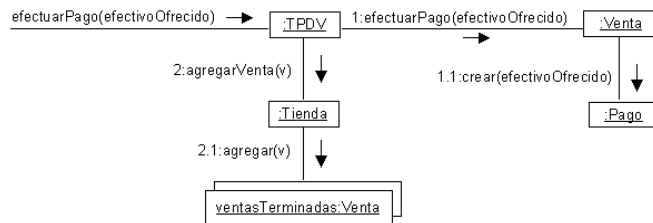
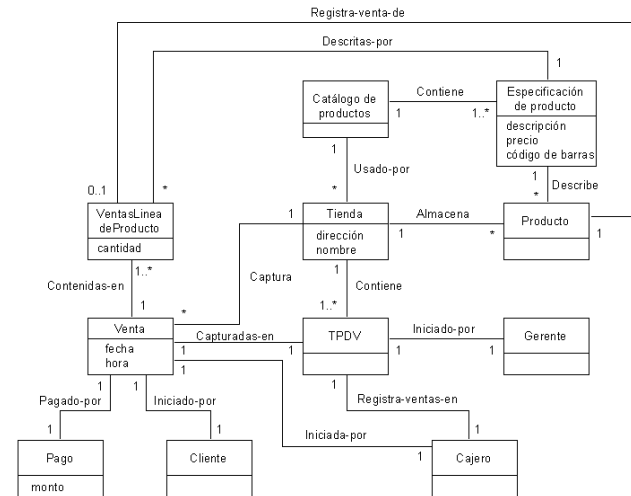
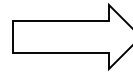
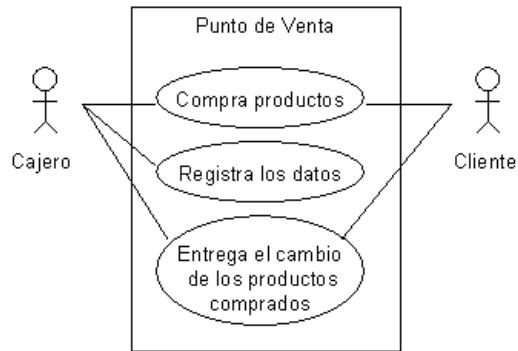
El diagrama de colaboración del caso de uso *pasarProducto* sería el siguiente:



Diagramas de clases



Análisis y Diseño OO



Análisis y Diseño OO

Nombre: Modelo-Vista-Controlador (MVC) [Buschmann 96].

Problema: Las interfaces de usuario son especialmente propensas a cambios de requerimientos. Cuando se extiende la funcionalidad de una aplicación, se deben modificar cosas, por ejemplo: menús, botones, ventanas, etc.

Solución: MVC divide una aplicación en tres áreas: procesamiento, entrada y salida. El componente *modelo* encapsula los datos y la funcionalidad principales de la aplicación. El componente *Vista* despliega la información al usuario, a partir de los datos del *Modelo*. Cada *Vista* tiene un componente *Controlador* asociado, que se encarga de recibir las entradas del usuario (movimiento del mouse, activación de los botones o entradas de teclado). Esta separación de componentes, permite, entre otras cosas, tener distintas vistas del mismo modelo.

Análisis y Diseño OO

Ejemplo: La mayoría de aplicaciones con interfaz gráfica utilizan este esquema. La programación con herramientas visuales como Visual Basic, JBuilder, Delphi, etc. sigue este esquema.

Beneficios: Es posible tener múltiples vistas de un mismo modelo. Es posible sincronizar las vistas cuando varios usuarios usan la misma aplicación (juegos multiusuario, sistemas colaborativos, etc). La separación conceptual permite intercambiar la vista y el controlador de un determinado modelo (*plug and play*).

Análisis y Diseño OO

El patrón MVC separa dos conceptos fundamentales en toda aplicación: la *interfaz* (vista y controlador) y el *modelo* (datos y funcionalidad).

Usando este patrón podríamos implementar la interfaz de nuestra aplicación de punto de venta como un applet Java, como un programa Java *stand-alone*, y de otras formas (no necesariamente en Java).

Fin

